

The Whole Story of Certified Quantum Randomness

A walk through the math, from the first roll of the dice
to a clean bag of certified random bits

A friendly reading guide. This document tells one continuous story: how a quantum computer’s output gets turned, step by step, into a guarantee of randomness anyone can verify. Each chapter introduces one new mathematical object, explains what it does in words, then writes down the formula and shows where it came from. By the end, every number in the protocol — including the headline “71,313 certified bits” from Liu et al. — will have a reason behind it. Nothing is skipped.

Prologue: the trust problem

Imagine you want some random numbers. Truly random ones — the kind a bank or a court or an election would trust. You can ask a quantum computer for them; quantum mechanics is genuinely unpredictable in a way no classical machine can fully imitate. But the quantum computer is owned by someone else, somewhere far away, and you have no way to look inside it. They could be running a real quantum machine. They could also be running a regular laptop pretending to be one, hoping you won’t notice.

What you need is a way to ask the machine some questions, see its answers, and decide — using only your own ordinary computer and a stopwatch — whether those answers really did come from a quantum measurement. And then, if they did, you need to convert those messy quantum outputs into a clean bag of perfectly random bits that you can hand to whoever needs them.

That conversion — from “messy bits out of an untrusted quantum box” to “clean, verified, certified-random bits with a receipt anyone can check” — is what this whole protocol does. It’s a chain of mathematical steps. Each step solves one specific worry. Let’s walk through them in order.

Cast of characters

The client. You. The one who wants random bits.

The server. The party who owns the quantum machine. *Untrusted by default.*

The verifier. A classical computer (also you, or anyone) that checks the server’s answers afterwards.

The challenge circuit. A specific quantum puzzle the server will be asked to solve.

n . The number of qubits in the puzzle. Liu et al. use $n = 56$. Our toy build uses $n = 8$.

$N = 2^n$. The number of possible answers (bitstrings) the puzzle can produce.

M . The number of *distinct* circuits used in one round of the protocol (each one different, each one used once). Liu et al.: $M = 30,010$.

1 Chapter 1: the puzzles we ask — Haar-random circuits

To check that the server is really doing quantum work, we don’t just ask it for “a random number.” We ask it to solve a specific kind of *quantum puzzle*, and we look at the pattern of its answers.

The puzzle is a **quantum circuit** — a sequence of gates acting on n qubits. Crucially, we don’t pick any old circuit. We pick one that is **as random as possible** — a so-called **Haar-random circuit**. “Haar” is just the name of the perfectly-uniform way to choose a random rotation in quantum space, the same way a fair coin is the perfectly-uniform way to choose a 50/50 outcome.

Picture shuffling a deck of cards. A Haar-random shuffle is the mathematically perfect shuffle, the one where every possible arrangement of the deck is equally likely. There is no bias, no pattern, no memory of the original order. We want our quantum circuits to be “shuffled” that thoroughly. (In practice, real circuits are only approximately Haar; they are close enough to behave like the ideal for our purposes.)

Why this matters: a perfectly random circuit is the hardest possible thing for a cheater to fake. There are no shortcuts a clever cheater could exploit, no patterns to learn. And, on the upside, a Haar-random circuit produces output statistics with a very specific, very recognizable shape — the shape we’ll use to detect cheating in the next chapter.

How we actually build the circuit: we don’t roll dice for hours. We start with a small, fresh, unpredictable **seed** (a short string of random bits), and we expand it deterministically into a full Haar-like circuit using a public, agreed-upon recipe (in code, a hash function called SHAKE-256). Two people with the same seed always get the same circuit. This matters because the verifier has to be able to reconstruct the exact same circuit the server saw, in order to check its answer.

So the picture so far: we drop a fresh small seed, expand it into M different Haar-random-like circuits, and ship them off to the server.

2 Chapter 2: the shape of quantum answers — Porter-Thomas

When a quantum computer runs one of these random circuits and measures its output, it doesn’t return “a random bitstring drawn uniformly.” It returns a bitstring drawn from a very specific, lopsided probability distribution.

Here’s what’s happening. Each of the $N = 2^n$ possible bitstrings has some probability $p(x)$ of coming out. For an honest random quantum circuit, these probabilities aren’t all equal. Some bitstrings are much more likely than others, and they’re spread out according to a specific law, called the **Porter-Thomas distribution**:

$$f(p) = N e^{-Np}$$

What this formula says: “most bitstrings have very low probability (near zero); a few have noticeably higher probability; and the spread between them follows an exponential curve.”

The intuition: imagine throwing N dots at random onto a line of length 1, then asking, “how big a piece of the line did each dot grab?” Most pieces would be small, a few would be lucky and large, and the distribution of sizes would look exponential. The output probabilities of a deep random quantum circuit behave exactly like that.

This shape is called *lopsided* because, unlike a flat uniform distribution where every outcome is equally likely, here the probability is concentrated unevenly across outcomes. The lopsidedness is the *fingerprint of real quantum behaviour* — a quantum circuit creates constructive and destructive interference between all the possible computational paths, and the resulting amplitudes pile up on some bitstrings while cancelling out on others. A classical machine that hasn’t actually done that interference produces a flat (uniform) distribution.

Two averages, easy to confuse

There are two different “averages” we can compute from this distribution, and they are different numbers. Getting them straight is important.

Average 1: “the average probability of a string”. If you list all N probabilities $p(x)$ and average them, you get

$$\frac{1}{N} \sum_x p(x) = \frac{1}{N} \cdot 1 = \frac{1}{N}.$$

This is forced — the probabilities must sum to 1, so their average is $1/N$. This is the *count-weighted* average; every bitstring counts equally.

Average 2: “the average probability of a string you actually sample”. When the quantum computer samples, it picks bitstring x *with probability* $p(x)$. So high-probability strings come up more often. The expected probability of a string you sample is

$$\sum_x p(x) \cdot p(x) = \sum_x p(x)^2.$$

Compute this using the Porter-Thomas density:

$$\mathbb{E}[p^2] = \int_0^\infty p^2 \cdot N e^{-Np} dp = \frac{2}{N^2}.$$

And so

$$\boxed{\sum_x p(x)^2 = N \cdot \frac{2}{N^2} = \frac{2}{N}.$$

So sampling lands you on strings averaging $2/N$, twice the count-weighted mean $1/N$.

This factor of 2 is the whole quantum signal: “a real quantum sampler lands on strings that are, on average, twice as probable as a random guess would predict.” It’s not a huge effect, but it is consistent and statistically detectable. The protocol is going to use exactly this gap.

3 Chapter 3: the cheater-detector — the F_{XEB} score

The server sends back its answers. The verifier looks at them and asks: “Do these look like genuine Porter-Thomas samples, or like flat noise, or like something a cheater cooked up?”

The tool for asking that question is called the **linear cross-entropy benchmark**, F_{XEB} . It is one number per batch of answers, and its job is to compare the server’s samples against the ideal Porter-Thomas prediction.

Here’s how it’s computed. Suppose the verifier received samples $x_1, x_2, \dots, x_m$ from m different circuits $C_1, \dots, C_m$. For each sample x_i , the verifier classically simulates the circuit C_i to compute the *ideal probability* $p_{C_i}(x_i)$ that an honest quantum machine would have assigned to that bitstring. Then:

$$F_{\text{XEB}} = \frac{N}{m} \sum_{i=1}^m p_{C_i}(x_i) - 1.$$

In plain English: take the probabilities the ideal quantum machine would assign to the strings the server actually returned; average those; multiply by N ; subtract 1.

What F_{XEB} means in four cases

The reason this formula is useful is that its expected value tells you, very cleanly, what’s going on.

1. **Uniform / flat / noise / spoof.** If the server returns bitstrings chosen uniformly at random (ignoring the circuit), then $\mathbb{E}[p_{C_i}(x_i)] = 1/N$, so

$$F_{\text{XEB}} = \frac{N}{m} \cdot m \cdot \frac{1}{N} - 1 = 1 - 1 = 0.$$

Uniform sampling gives **score 0**.

2. **Perfect quantum.** If the server is a perfect quantum computer sampling exactly from the ideal distribution, then $\mathbb{E}[p_{C_i}(x_i)] = \sum_x p(x)^2 = 2/N$, so

$$F_{\text{XEB}} = \frac{N}{m} \cdot m \cdot \frac{2}{N} - 1 = 2 - 1 = 1.$$

Perfect quantum sampling gives **score 1**.

3. **Real noisy quantum.** A real quantum device is partly perfect, partly noisy, with a *fidelity* $\phi \in (0, 1)$ that measures how close to ideal it is. In this case

$$F_{\text{XEB}} \approx \phi.$$

A real machine gives a score between 0 and 1; Liu et al.’s threshold χ for acceptance is set somewhere in this band ($\chi = 0.3$ in the paper).

4. **Cherry-picking cheater.** A clever cheater could classically compute the high-probability strings for each circuit (cheaper than full simulation in some regimes) and submit *only* those. The strings they return have $p_{C_i}(x_i) \gg 2/N$, so

$$F_{\text{XEB}} \gg 1.$$

A cherry-picking attack gives a **suspiciously high score**, ≥ 1 .

A precision worth pinning down here. Liu et al. themselves use a one-sided check: the round passes iff $F_{XEB} \geq \chi$. This works inside their security framework because their adversary model is compute-rational — the worst-case cheater wants to just barely clear the threshold using as few honest quantum rounds as possible, so they won’t score $F_{XEB} \gg 1$ (which would require more quantum content, not less). The certification bound $Q_{\min} = \min\{Q : \varepsilon_{adv}(Q, \chi) \geq 4\varepsilon_s\}$ is built around this one-sided test.

Our implementation (from v0.1 onward) goes one step further and enforces a two-sided check

$$\chi_{low} \leq F_{XEB} \leq \chi_{high}$$

as a defence-in-depth measure against an adversary outside Liu’s model — one that wastefully scores $\gg 1$, or that somehow circumvents the compute-budget assumption and tries pure heavy-output generation. This is a strictly more conservative variant than Liu requires, adopted deliberately. The lower bound is the one that’s load-bearing; the upper bound catches a different threat class than Liu’s adversary model contemplates.

So we now have a single number that, in expectation, separates the four big possibilities: cheater-by-noise (≈ 0), real machine ($\approx \phi$), perfect machine (1), cheater-by-cherrypicking ($\gg 1$). Pass-fail.

Why one shot per circuit matters

There’s a subtle but important design choice. The protocol uses M *distinct* circuits, each measured *once*. Why not measure one circuit many times instead?

Two reasons. First, the security argument turns into one neat one-shot-per-circuit lemma, instead of a complicated many-shots-on-one-circuit argument that’s prone to amortization attacks (where a cheater simulates the one circuit once and then “cheaply” returns many fake samples from the same cache). Second, on trapped-ion hardware like Quantinuum H2, the per-circuit overhead is small, so doing M one-shot circuits is comparable in time to doing 1 circuit with M shots — and it’s much harder to spoof.

4 Chapter 4: the clock — why timing is half the proof

A high F_{XEB} score by itself doesn’t prove quantumness. A patient cheater with a big enough classical supercomputer could simulate every circuit, compute the answers, and submit good ones. They would pass any score check. So what makes them *infeasible*?

The deadline. The server has only a short window (Liu et al.: a per-circuit response time of roughly a few seconds) to return each sample. A genuine quantum machine takes this time naturally — it just measures. A classical simulator, on a 56-qubit circuit, would take *many hours* per circuit. The deadline closes off the classical route.

So the proof of quantumness has two halves, and *neither half alone is enough*:

- **High F_{XEB}** alone rules out “fast but fake” (random junk).
- **Fast response** alone rules out “correct but slow classical” (the patient simulator).
- **Both together** can only be done by a real quantum computer. Correct *and* fast.

Worth keeping in your back pocket: the verifier’s classical check (simulating the circuits to compute the ideal probabilities) is itself slow — it takes hours or days. That’s fine, because the verifier isn’t on the deadline; only the server is. The asymmetry between “the server is timed but the verifier isn’t” is what makes the protocol work: the expensive check is allowed to be expensive, because that expense doesn’t help the cheater.

The deadline doesn’t just enable the proof; it sets a number called the **adversary’s faking budget**. Let A be the adversary’s compute power (in floating-point operations per second), $t_{\text{threshold}}$ the deadline, and B the classical cost (in operations) of faking one circuit’s answer well. Then the maximum number of circuits a cheater could plausibly fake within the budget is

$$\Phi_A \approx \frac{A \cdot t_{\text{threshold}}}{B}.$$

This number is going to drive everything in the next chapter.

5 Chapter 5: the worst-case floor — $Q_{\min}$

We now have the two pieces we need to start counting how much real quantum work *must* have happened: a score, F_{XEB} , that the server passed; and the adversary’s faking budget, Φ_A , set by the deadline.

Here is the central question, which Liu et al.’s Lemma 2 (a refinement of an earlier result of Aaronson and Hung) answers. Suppose the server returned M samples and they passed the score check. Some of them *might* have been faked by a clever cheater within their compute budget — but not all of them. Some had to be genuine quantum measurements. How many, in the worst case?

The answer is a single number called $Q_{\min}$ — the **minimum number of circuits** that must have been faithfully quantum, given that the batch passed the test, even if we assume the most capable cheater possible.

The derivation is technical (it’s a hypergeometric-tail optimization with a smoothing parameter ε_2 , looking for the worst-case δ -fraction of forced-quantum rounds consistent with the observed score), but the moral is simple: **the more the score exceeds the cheater’s fakable score, the higher the $Q_{\min}$ floor.** A cheater can fake some circuits well enough to inflate the average score, but not so many that the whole batch hits Liu et al.’s threshold. The remainder — the forced-quantum part — is $Q_{\min}$.

For Liu et al.’s published parameters ($n = 56$, $M = 30,010$, $\chi = 0.3$, deadline ~ 2 s, adversary at 1.1×10^{18} FLOPS), the paper reports

$$Q_{\min} = 1297.$$

That is: out of 30,010 circuits, at least 1297 *must* have been honest quantum measurements. The rest *could* have been faked. We don’t get to know *which* of the 30,010 were genuine — we only know that at least 1297 of them were.

One honest note. The exact derivation of $Q_{\min}$ from the published parameters depends on a few conventions (the precise definition of Φ_A , the optimization grid, the ε_2 tail) that live in the authors’ code, not in the

paper. Our own implementation reproduces the entropy and extraction downstream of $Q_{\min}$ exactly, but our $Q_{\min}$ derivation is currently only correct to order of magnitude. This is a known and deliberately flagged calibration item, not a math gap.

6 Chapter 6: how much randomness do we have? — certified min-entropy

We now know that at least $Q_{\min}$ of the server’s samples are genuine. The next question is: *how much randomness is in each one of those genuine samples?*

The answer is determined by the Porter-Thomas distribution. For the kind of randomness we’ll certify — randomness that’s resistant to a sophisticated adversary, including one with quantum side information — the right measure is not Shannon entropy but **collision entropy** (also called the 2-Rényi entropy), defined as

$$H_2(P) = -\log_2 \sum_x p(x)^2.$$

We have already computed $\sum_x p(x)^2$ for the Porter-Thomas distribution. It is $2/N$. Substituting:

$$H_2 = -\log_2 \left(\frac{2}{N} \right) = \log_2 N - \log_2 2 = n - 1.$$

Per-round collision entropy: $H_2 = n - 1$ bits.

For Liu et al.’s $n = 56$, that is 55 bits of randomness per honest round.

One way to feel the $n-1$: a perfectly uniform distribution over N strings would have $\sum p(x)^2 = N \cdot (1/N)^2 = 1/N$, giving $H_2 = n$. The Porter-Thomas distribution has twice that sum-of-squares ($2/N$ instead of $1/N$), because it’s lopsided. The factor of 2 inside the log becomes a single bit lost outside. So $n-1$ is exactly “ n minus the one-bit cost of Porter-Thomas’s lopsidedness.”

So we have $Q_{\min}$ forced-quantum rounds, each contributing $n - 1$ bits of guaranteed unpredictability. Multiplying gives the total: $Q_{\min} \cdot (n - 1)$ raw bits.

But we don’t get to keep all of these as certified bits. The protocol has a target probability of failure — a number called the *smoothing parameter* ε_s , the tiny chance that the certification is wrong by accident or by adversarial luck. To get a clean, formal bound on the entropy, we have to pay a small price in bits:

$$H_{\min}^{\text{cert}} = Q_{\min} \cdot (n - 1) - \log_2 \left(\frac{1}{\varepsilon_s} \right).$$

This is Liu et al.’s Theorem 1.

Plugging in the paper’s numbers

For Liu et al., $Q_{\min} = 1297$, $n = 56$, and $\varepsilon_s = \varepsilon_{\text{sou}}/4 = 2.5 \times 10^{-7}$:

$$H_{\min}^{\text{cert}} = 1297 \cdot 55 - \log_2\left(\frac{1}{2.5 \times 10^{-7}}\right) = 71,335 - 21.9 \approx \boxed{71,313 \text{ bits}}.$$

That is the paper’s headline number: *at least 71,313 bits of provably-unpredictable randomness* sit somewhere inside the $M \cdot n = 1,680,560$ raw bits the server returned.

The trouble is, we don’t know *where* the unpredictability is. It’s mixed with up to 1,600,000-plus predictable or biased bits. The raw output is not yet usable. We need to purify it.

7 Chapter 7: turning ore into gold — Toeplitz extraction

Here is the analogy that ties everything together. The server’s raw quantum output is like raw **gold ore**. It does contain gold (the 71,313 bits of certified entropy), but the gold is mixed with rock (the predictable, biased, or partly-known bits). You can’t give someone raw ore and call it gold. You have to refine it.

The refinement step — the refinery — is called **randomness extraction**. It takes the raw bits plus a small extra seed (an independent random value, in our protocol a fresh pulse from the **drand** beacon), and produces an output that’s statistically *indistinguishable from uniform*, even to an adversary with quantum side-information.

The specific extractor we use is the **Toeplitz extractor**: a deterministic linear map over $\text{GF}(2)$ (the field of bits, with arithmetic mod 2). Pick a Toeplitz matrix T of size $\ell \times (M \cdot n)$ — a matrix whose entries are determined by its first row and first column, which makes it efficient to store and multiply. Seed the entries with a fresh independent random source. Then compute

$$\boxed{\text{output} = T \cdot \text{raw_bits} \pmod{2}.$$

The output is an ℓ -bit string.

How large can ℓ be? The **Leftover Hash Lemma** (a foundational result in cryptography) tells us. To produce output that is ε_{sou} -close to uniform against an adversary with quantum side information, we can extract up to

$$\boxed{\ell \leq Q_{\min} \cdot (n - 1) - 3 \log_2\left(\frac{1}{\varepsilon_{\text{sou}}}\right) - 2 \text{ bits.}}$$

This is Liu et al.’s Corollary 7.

Where the formula’s pieces come from

- $Q_{\min} \cdot (n - 1)$ — the raw entropy budget, same as before.
- $-3 \log_2(1/\varepsilon_{\text{sou}})$ — the price of a *quantum-secure* extractor. Against a purely classical adversary the price would be $2 \log_2(1/\varepsilon_{\text{sou}})$; the extra factor (the 3) buys robustness against a quantum side-information attacker.
- -2 — a small additive constant from the LHL bound’s exact statement.

Plugging in the paper’s numbers, again

For $\varepsilon_{\text{sou}} = 10^{-6}$:

$$\ell \leq 1297 \cdot 55 - 3 \log_2(10^6) - 2 = 71,335 - 59.8 - 2 \approx \boxed{71,273 \text{ bits}}.$$

This is Liu et al.’s second published number: *71,273 bits of clean, uniform, certified randomness can be extracted from the batch.*

The difference $71,313 - 71,273 = 40$ bits is the **cost of the refinery**: ~ 40 bits of certified entropy are spent on margins to guarantee that the output is statistically uniform even against a quantum-smart adversary. About 0.06% of the gold is lost in refining. Modest, but real.

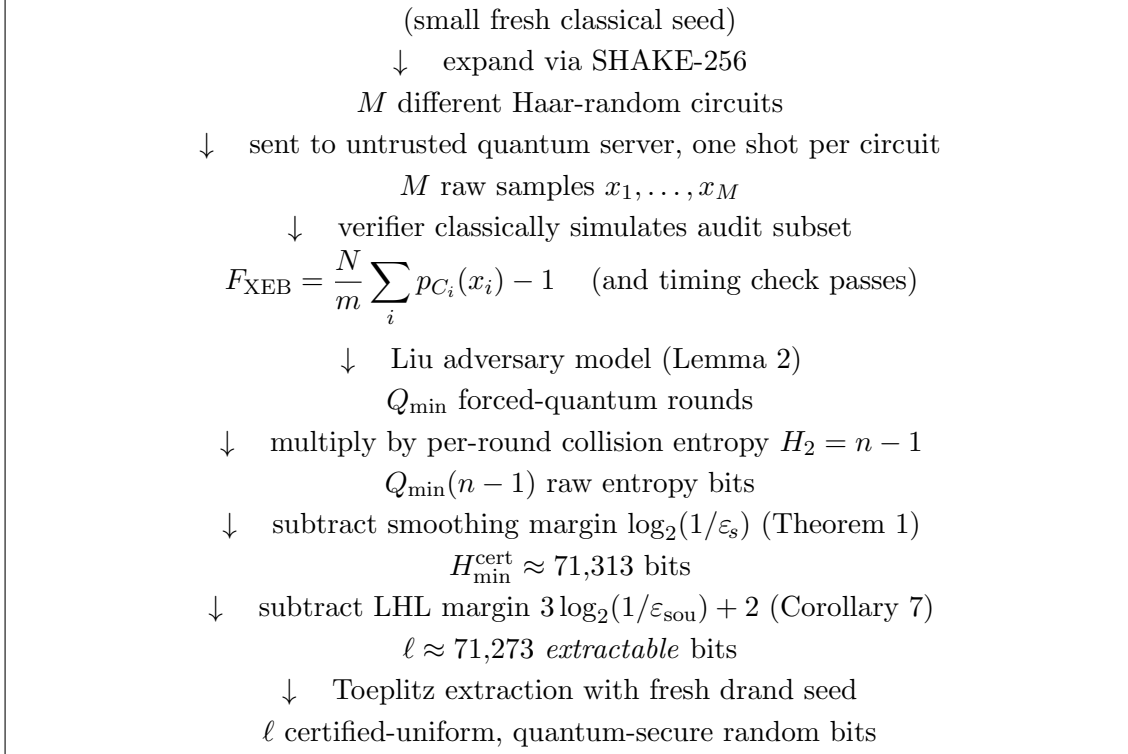
8 Chapter 8: the audit trail

There is one more leg, less mathematical but no less important: every step of the run — the seed used, the circuits generated, the samples returned, the score computed, the $Q_{\min}$ value, the extractor seed, the final output — is recorded in a tamper-evident audit log (hash-chained, so any after-the-fact modification breaks the chain). Anyone can later replay the log and re-verify that the round was honest end-to-end: same circuits from the same seed, same score from the same samples, same final bits from the same extractor.

This is what makes the randomness *publicly verifiable*, not just secretly believable. The client and the server and an outside auditor all see the same record.

Epilogue: the full chain, in one diagram

Putting it all together, the math chain looks like this:



Every arrow in that diagram is one of the formulas we walked through:

- Porter-Thomas: $f(p) = Ne^{-Np}$, and $\sum_x p(x)^2 = 2/N$.
- $F_{\text{XEB}} = (N/m) \sum_i p_{C_i}(x_i) - 1$.
- Cases: noise $\rightarrow 0$, real $\rightarrow \phi$, perfect $\rightarrow 1$, cherry-pick $\rightarrow \gg 1$.
- Adversary budget: $\Phi_A = A \cdot t_{\text{threshold}}/B$.
- Collision entropy of PT: $H_2 = -\log_2(2/N) = n - 1$.
- Certified entropy (Liu Thm 1): $H_{\min}^{\text{cert}} = Q_{\min}(n - 1) - \log_2(1/\varepsilon_s)$.
- Extracted bits (LHL, Cor. 7): $\ell \leq Q_{\min}(n - 1) - 3 \log_2(1/\varepsilon_{\text{sou}}) - 2$.
- Toeplitz: **out** = $T \cdot \mathbf{raw} \pmod{2}$.

What's solid, and what's still owed

In the spirit of being honest about which links of this chain are nailed down and which are still being polished:

Solid, derived from first principles. The collision entropy $H_2 = n - 1$ comes straight from the Porter-Thomas integral. Plugged into the certified-entropy and extraction formulas with the paper's $Q_{\min} = 1297$, our code reproduces the paper's 71,313 and 71,273 *exactly*. A regression test locks these in so any future change that breaks the math fails the test the next time anyone runs it.

Still owed. The *derivation* of $Q_{\min}$ from the adversary model is the one upstream piece we currently reproduce only to order of magnitude. The exact conventions used in the paper (the precise form of Φ_A , the optimization grid, the ε_2 tail) live in the authors’ supplementary code. Calibrating against those conventions is a bounded follow-up; we deliberately refuse to back-fit parameters to force $Q_{\min} = 1297$, because that would be fitting to the answer rather than reproducing it. The regression test marks this case as an honest, expected-to-fail (**xfail**) item.

One last picture, for the very end

Imagine the whole protocol as a small factory. The seed is the ignition. The server is a foundry that smelts ore (raw quantum samples) according to recipes it can’t see in advance. The verifier is the assayer, deciding how much gold was in the ore. The extractor is the refinery, melting the assayed ore into gold bars. The audit log is the ledger that lets any customer trace each bar back to its ore and its assay. And the deadline is the timed conveyor belt: a real foundry can keep up; a forger pretending to be one cannot.

That is the entire story. Every number a customer ever sees — “71,273 certified random bits, with $\varepsilon_{\text{sou}} = 10^{-6}$, on $n = 56$ qubits” — comes from a chain of clear, derivable, checkable steps, each one designed to close a specific worry about a specific way the protocol could fail. Nothing magic, nothing mysterious: just one careful step after another, from a small seed of trust to a clean bag of certified gold.